

Ex03 – Cryptographic Hash Functions (Resumo Explicativo e Simplificado)

1. O que é uma função hash criptográfica

Uma **função hash criptográfica** é um **algoritmo que transforma dados de qualquer tamanho** (texto, ficheiro, etc.) **numa sequência de tamanho fixo** chamada **digest** (ou resumo). O resultado é único para cada entrada, como uma impressão digital dos dados.

Exemplo: SHA256 gera sempre uma saída de **256 bits** (64 caracteres hexadecimais), independentemente do tamanho do texto original. *(32 bytes)*

2. Propriedades de uma boa função hash

- **1. Determinística:** o mesmo input gera sempre o mesmo output.
- **2. Avalanche effect:** uma **pequena mudança no input causa uma grande diferença no output.**
- **3. Irreversibilidade:** não é possível recuperar o texto original a partir do hash.
- **4. Uniformidade:** as saídas são distribuídas de forma aleatória.
- **5. Resistência a colisões:** é extremamente improvável que dois inputs diferentes gerem o mesmo hash.

3. Testar hashes no terminal

No Linux, podes usar os comandos abaixo para gerar hashes de um texto:

- **md5sum** — gera hash MD5 (**128 bits**).
- **sha256sum** — gera hash SHA-256 (**256 bits**).
- **sha384sum** — gera hash SHA-384 (**384 bits**).
- **sha512sum** — gera hash SHA-512 (**512 bits**).

Exemplo:

\$ echo -n 'Seguranca Informatica' | sha256sum

```
paulo@paulo-Vivobook-ASUSLaptop-M6500QE-M6500QE:~$ echo "OLA" | md5sum
f5648215d27762b04a5eeb17823b9f77 -
paulo@paulo-Vivobook-ASUSLaptop-M6500QE-M6500QE:~$ echo -n "OLA" | md5sum
cc5214661e49a3fb1b02fa250aa0fdac -
```



O parâmetro **-n** evita que o comando **echo** adicione uma quebra de linha (**'\n'**) ao final do texto, garantindo que o hash seja calculado apenas sobre o texto desejado.

4. Avalanche Effect (Efeito Avalanche)

O **efeito avalanche** significa que uma **pequena alteração na entrada** — até mesmo um único bit — **muda drasticamente o resultado do hash**. Este comportamento é essencial para a segurança de uma função hash.

Para verificar isto, calcula-se o hash de duas mensagens que diferem num único caractere e compara-se o número de bits diferentes entre os dois hashes (chamado **Hamming Distance**).

Dica: caracteres '0' e '1' diferem em apenas um bit (ver tabela ASCII).

5. Análise estatística do efeito avalanche

O exercício pede para criar um programa (em C ou Python) que gere múltiplas mensagens aleatórias com apenas um bit alterado e calcule o número médio de bits diferentes entre os hashes. Isso serve para confirmar a distribuição aleatória das mudanças no hash.

- **Ferramentas:** OpenSSL em C ou módulo cryptography em Python.
- **Comando C:** gcc avalanche-analysis.c -o avalanche-analysis -lcrypto
- **Comando Python:** sudo apt install python3-cryptography
- **Visualização:** usar gnuplot para criar um gráfico de distribuição.

6. Proof-of-Work (Prova de Trabalho)

A prova de trabalho é um mecanismo que demonstra que uma **tarefa custosa em termos de tempo/computação foi realizada**. Ela é usada em sistemas como o Bitcoin para validar blocos.

No exercício, o objetivo é encontrar uma frase que, ao ser passada por uma função hash (como SHA-256), gere um resultado que comece com um certo número de zeros (ex: três zeros em hexadecimal).

Exemplo:

```
$ echo -n 'My student number is 1234 and I love crypto!' | sha256sum
```

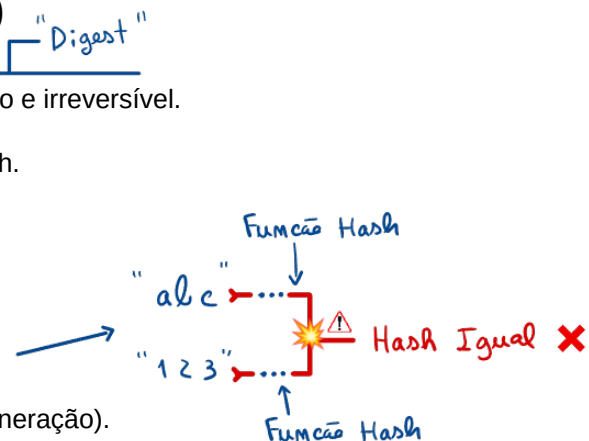
Para encontrar um hash com três zeros iniciais, é preciso tentar várias combinações — o que representa o trabalho computacional exigido.

7. Aplicações das funções hash

- Verificação de integridade de ficheiros (md5sum, sha256sum).
- Armazenamento seguro de senhas (hash + salt).
- Assinaturas digitais e certificados SSL.
- Provas de trabalho em blockchains.

8. Perguntas possíveis (e respostas curtas)

- O que é uma função hash criptográfica?
→ Um algoritmo que transforma dados em um resumo único e irreversível.
- O que é o efeito avalanche?
→ Uma pequena mudança no input altera totalmente o hash.
- O que é a Hamming Distance?
→ Número de bits diferentes entre dois hashes.
- Qual o tamanho da saída do SHA-256?
→ 256 bits (64 caracteres hexadecimais).
- O que é uma colisão?
→ Quando dois inputs diferentes produzem o mesmo hash.
- Para que serve a prova de trabalho?
→ Mostrar que foi realizado um cálculo dispendioso (ex: mineração).
- O que faz o comando echo -n?
→ Evita adicionar uma quebra de linha no texto antes de gerar o hash.



Preparação Prática – Cryptographic Hash Functions (Guia 3)

Versão expandida com perguntas abertas e de preencher código. Respostas no fim.

Parte 1 – Exercícios (sem respostas)

1) Hashes na linha de comando (md5sum, sha256sum, sha384sum, sha512sum)

a) Produz uma hash sem newline final
echo -n "The quick brown fox" | sha256sum

b) Com newline (diferente!)
echo "The quick brown fox" | sha256sum

c) Frases semelhantes para testar avalanche
echo -n "abc" | sha512sum
echo -n "abC" | sha512sum

Q1.1 Explique por que usamos o parâmetro -n no echo. O que muda se o omitir?

Q1.2 A hash imprime o número de caracteres do digest? Justifique com sha256 vs sha512.

Q1.3 Dê dois exemplos de tamanhos de entrada muito diferentes que geram hashes de mesmo tamanho.

Vai cifrar também o /n
Em hexadecimal
1 car - 2 bytes
2 - 32
64 caracteres
sha256 - 32 bytes
sha512 - 64 bytes
↓
128 caracteres

2) Efeito Avalanche – alteração de 1 bit (C)

```
/* avalanche_bit.c */  
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
#include <time.h>  
#include <stdint.h>  
#include <openssl/sha.h>
```

```
static int popc8(uint8_t x){ return __builtin_popcount(x); }
```

```
int main(int argc, char **argv){  
    if(argc!=3){ fprintf(stderr,"Uso: %s <bytes> <N>\n", argv[0]); return 1; }  
    int bytes = atoi(argv[1]);  
    int N = atoi(argv[2]);  
    if(bytes<=0 || N<=0){ fprintf(stderr,"args inválidos\n"); return 1; }
```

```
    unsigned char *m = malloc(bytes);  
    if(!m){ perror("malloc"); return 1; }  
    srand((unsigned)time(NULL));  
    for(int i=0;i<bytes;i++) m[i] = rand() & 0xFF;
```

```
    unsigned char h0[SHA256_DIGEST_LENGTH], h1[SHA256_DIGEST_LENGTH];
```

```
    /* PREENCHA: calcular hash inicial h0 da mensagem m */
```

```
    for(int k=0;k<N;k++){
```

```
        unsigned char *c = malloc(bytes);
```

```
        memcpy(c, m, bytes);
```

```
        int bit = rand() % (bytes*8);
```

```
        c[bit/8] ^= (1u << (bit%8));
```

↓
Os hashes são gerados
sempre com o mesmo tamanho

→ SHA256(m, bytes, h0)
dados entrada | n° de bytes | buffer de saída

```

    SHA256(c, bytes, h1);

    int hd=0;
    for(int j=0;j<SHA256_DIGEST_LENGTH;j++) hd += popc8(h0[j]^h1[j]);
    printf("%d 1\n", hd);
    free(c);
}
free(m);
return 0;
}

```

Q2.1 Preencha a linha que falta para calcular a hash inicial (função OpenSSL correta).

Q2.2 Explique como garantir que a mesma mensagem alterada 1-bit não se repete.

3) Avalanche – alteração de 1 byte (C)

```

/* avalanche_byte.c */
// ... base igual ao anterior
/* PREENCHA apenas a parte da alteração do byte:
    int pos = rand() % bytes;
    unsigned char mask = (rand() % 255) + 1; // 1..255
    c[pos] ^= mask;
*/

```

1
Assim como os bits já
utilizados numa lista e fazer
a comparação com o próximo
bit doo, se pertencer, não
alterar

Q3.1 Preencha a lógica de alteração de 1 byte (sem repetir estados).

Q3.2 Compare as distribuições (1 bit vs 1 byte). O que espera teoricamente?

4) Distância de Hamming entre dois digests hex (C)

```

/* hamming_hex.c */
#include <stdio.h>
#include <string.h>
#include <stdint.h>
#include <ctype.h>

static int hex2nib(char c){
    if(c>='0'&&c<='9') return c-'0';
    c = tolower((unsigned char)c);
    if(c>='a'&&c<='f') return 10 + (c-'a');
    return -1;
}

int main(int argc, char **argv){
    if(argc!=3){ fprintf(stderr,"Uso: %s <hex1> <hex2>\n", argv[0]); return 1; }
    const char *a=argv[1], *b=argv[2];
    int la=strlen(a), lb=strlen(b);
    if(la!=lb || la%2){ fprintf(stderr,"tamanhos inválidos\n"); return 1; }

    int bits=0;
    for(int i=0;i<la;i+=2){
        int a1=hex2nib(a[i]), a2=hex2nib(a[i+1]);
        int b1=hex2nib(b[i]), b2=hex2nib(b[i+1]);
        if(a1<0||a2<0||b1<0||b2<0){ fprintf(stderr,"hex inválido\n"); return 1; }
        uint8_t x = (uint8_t)((a1<<4)|a2);
        uint8_t y = (uint8_t)((b1<<4)|b2);
        bits += __builtin_popcount((unsigned)(x^y));
    }
    printf("%d\n", bits);
}

```

Se alterarmos 1 byte a
alteração vai ser mais dispersa
mas ambos não alteram
+/- 128 bits, por causa do
efeito avalanche

```

    return 0;
}

```

Q4.1 Complete a função hex2nib e explique a verificação de tamanhos.

5) Proof-of-Work (C) — prefixo com 3 zeros hex

```

/* pow3.c */
#include <stdio.h>
#include <string.h>
#include <stdint.h>
#include <openssl/sha.h>

int main(int argc, char **argv){
    if(argc<2){ fprintf(stderr,"Uso: %s <base_string>\n", argv[0]); return 1; }
    const char *base = argv[1];
    unsigned long nonce = 0;
    unsigned char buf[1024], d[32];
    for(;;){
        int n = snprintf((char*)buf, sizeof(buf), "%s%lu", base, nonce);
        SHA256(buf, n, d);
        if(d[0]==0 && (d[1]>>4)==0){
            for(int i=0;i<32;i++) printf("%02x", d[i]);
            printf(" nonce=%lu\n", nonce);
            break;
        }
        nonce++;
    }
    return 0;
}

```

Q5.1 Qual a complexidade esperada (número médio de tentativas) para 3 zeros hex?

Q5.2 Adapte o código para 4 zeros hex. Que condição bit a bit deve verificar?

6) PoW em lote — média de tentativas (C)

```

/* pow_batch.c */
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <openssl/sha.h>

int main(int argc, char **argv){
    if(argc!=3){ fprintf(stderr,"Uso: %s <base> <M>\n", argv[0]); return 1; }
    const char *base = argv[1];
    int M = atoi(argv[2]);
    unsigned long total=0;
    for(int i=0;i<M;i++){
        unsigned long nonce=0; unsigned char d[32]; char in[256];
        for(;;){
            int n = snprintf(in, sizeof(in), "%s_%d_%lu", base, i, nonce);
            SHA256((unsigned char*)in, n, d);
            if(d[0]==0 && (d[1]>>4)==0){ total += nonce; break; }
            nonce++;
        }
    }
    printf("media=%.2f\n", (double)total/M);
    return 0;
}

```

Porque estamos à espera de uma condição específica e não colisão

Q6.1 Explique em que sentido o paradoxo do aniversário se aplica ou não a esta média.

↳ Semelhante Total

7) Visualização com gnuplot

```
# Histograma via gnuplot
./avalanche_bit 1024 1000 | gnuplot -p -e "plot '-' using 1:2 with boxes"
```

Q7.1 Qual o formato de saída mínimo exigido para produzir o histograma diretamente?

8) Teoria curta

Q8.1 Defina resistência a preimagem, segunda preimagem e colisão.

Duas msg diferentes podem gerar o mesmo Hash

Q8.2 Explique por que MD5 não é apropriado para aplicações de segurança modernas.

Q8.3 Em SHA-256, qual a esperança de Hamming distance quando 1 bit é alterado no input?

É esperado ser o dobro, pelo facto do efeito avalanche

9) Preencher código – imprimir SHA-256 em hex (C)

```
/* sha_hex.c */
#include <stdio.h>
#include <string.h>
#include <openssl/sha.h>

int main(void){
    unsigned char buf[4096], d[32];
    size_t n = fread(buf, 1, sizeof(buf), stdin);
    /* PREENCHA: calcular o SHA-256 de buf[0..n) para d */
    /* PREENCHA: imprimir d como 64 chars hex, sem espaços */
    return 0;
}
```

Q9.1 Preencha as duas linhas em falta.

10) Comparar SHA-256 vs SHA-512 (C, esqueleto)

```
/* sha_cmp.c */
#include <stdio.h>
#include <string.h>
#include <openssl/sha.h>

int main(void){
    const char *msg = "Test message";
    unsigned char d256[SHA256_DIGEST_LENGTH];
    unsigned char d512[SHA512_DIGEST_LENGTH];
    /* PREENCHA: SHA256(msg, strlen(msg), d256); */
    /* PREENCHA: SHA512(msg, strlen(msg), d512); */
    printf("len256=%d, len512=%d\n", (int)SHA256_DIGEST_LENGTH, (int)SHA512_DIGEST_LENGTH);
    return 0;
}
```

Q10.1 Preencha as chamadas. Comente a diferença de tamanho do digest.

Parte 2 – Respostas

Q1.1: echo -n evita adicionar '\n' ao fim; sem -n o input muda e a hash muda.

Q1.2: Não. 256/512 referem-se ao tamanho do digest (bits), não ao length do input.

Q1.3: Ex.: string vazia e ficheiro grande; ambas dão 256 bits em SHA-256.

Q2.1: SHA256(m, bytes, h0);

Q2.2: Gerar posições sem reposição (track de bits usados) ou sortear até encontrar inédito.

Q3.1: pos = rand()%bytes; mask=(rand()%255)+1; c[pos]^=mask; Evitar repetir o mesmo (pos,mask).

Q3.2: Distribuições centradas em ~128 bits; diferenças pequenas de variância entre bit/byte flip.

Q4.1: hex2nib mapeia [0-9a-fA-F]→0..15; exigir la==lb e la par (2 hex por byte).

Q5.1: $1/16^3 = 1/4096$, logo ~4096 tentativas em média.

Q5.2: 4 zeros: d[0]==0 && d[1]==0 (ou 16 zeros em bits: 0 primeiros 16 bits).

Q6.1: A média de tentativas para prefixo fixo não usa colisões; o paradoxo do aniversário é irrelevante aqui.

Q7.1: Linhas '1' por ocorrência; gnuplot agrega e plota 'with boxes'.

Q8.1: Preimagem: dado y, difícil achar x c/ $H(x)=y$; 2ª preimagem: dado x, achar $x' \neq x$ c/ $H(x')=H(x)$; colisão: achar $x \neq x'$ com mesma hash.

Q8.2: MD5 tem colisões práticas conhecidas; inseguro para integridade e assinaturas.

Q8.3: ~ metade dos 256 bits → ~128 bits esperados.

Q9.1: SHA256(buf, n, d); for(i=0;i<32;i++) printf("%02x", d[i]);

Q10.1: SHA256(msg, strlen(msg), d256); SHA512(msg, strlen(msg), d512); 32 vs 64 bytes (256 vs 512 bits).